

CSS III — Grid, position, responsive y composición de interfaces completas

En esta sesión

En esta sesión vas a dar un paso importante: dejarás de pensar solo en elementos sueltos o pequeños bloques alineados y empezarás a trabajar la **composición completa de una interfaz**, incluyendo su adaptación a distintos tamaños.

Vas a estudiar:

- CSS Grid como sistema de layout bidimensional;
- reparto de espacio con `fr`, `repeat()` y `minmax()`;
- rejillas flexibles con `auto-fit` y `auto-fill`;
- composición con `grid-template-areas`;
- posicionamiento con `position`;
- control de capas con `z-index`;
- tratamiento de imágenes dentro del layout;
- responsive design, media queries y enfoque mobile-first;
- y una primera aproximación a container queries.

Esta sesión es importante porque aquí CSS deja de ser solo una forma de “alinear cosas” y pasa a convertirse en una herramienta para **estructurar pantallas completas, adaptarlas y mantenerlas coherentes**.

1. Introducción

En la sesión anterior trabajaste selectores más precisos, pseudo-clases, pseudo-elementos, box model, `overflow`, `display`, Flexbox y una primera capa de calidad de código. Eso te permitió resolver muchos problemas reales de componentes y alineación en una sola dimensión.

Pero una interfaz completa suele necesitar algo más.

Muy pronto aparecen estructuras como estas:

- una cabecera arriba y un contenido principal debajo;

- una barra lateral y una zona central;
- una rejilla de tarjetas que cambia según el ancho disponible;
- una portada con varias zonas;
- un dashboard con módulos de tamaños distintos;
- una composición general donde filas y columnas importan al mismo tiempo.

Ahí es donde **Grid** empieza a tener mucho sentido.

Idea clave

Flexbox funciona especialmente bien cuando organizas elementos en **una dimensión**.
Grid destaca cuando necesitas pensar la interfaz en **filas y columnas al mismo tiempo**.

Además, una interfaz actual no se resuelve solo en un ancho de pantalla. También debe adaptarse con criterio. Eso significa que, en esta sesión, layout y responsive van unidos: no tiene demasiado sentido aprender a construir una rejilla si luego no sabes cómo debe comportarse al cambiar el espacio disponible.

También entra aquí una cuestión importante: no todo es Grid. Cuando empiezas a componer pantallas reales, también necesitas saber:

- cuándo usar `position`;
- cómo se superponen elementos;
- cómo tratar imágenes dentro de tarjetas y bloques;
- y cómo evitar que una solución visual rompa la claridad estructural del HTML.

Referencias del apartado 1

- MDN Web Docs. *CSS grid layout*.
- MDN Web Docs. *Basic concepts of grid layout*.
- MDN Web Docs. *position*.
- MDN Web Docs. *Responsive design*.

2. Grid como sistema de layout

Grid es un sistema de layout bidimensional. Eso significa que puede trabajar con columnas y filas al mismo tiempo.

Para activar Grid en un contenedor:

```
.layout {  
  display: grid;  
}
```

A partir de ese momento, sus hijos pasan a colocarse dentro de una rejilla.

Esta idea es importante: con Grid no estás “alineando cajas una detrás de otra”, sino definiendo una estructura espacial que organiza el contenido.

Eso te permite pensar mejor en:

- cuántas columnas tiene una zona;
- cómo se reparten;
- qué separación hay entre bloques;
- cómo crecen o se reducen;
- y qué hueco ocupa cada parte de la interfaz.

gap

Igual que ocurría con Flexbox, `gap` es una herramienta muy útil para separar elementos sin repartir márgenes manualmente.

```
.layout {  
  display: grid;  
  gap: 20px;  
}
```

🔥 Buena práctica

Cuando el espacio entre filas y columnas forma parte de la rejilla, `gap` suele ser más claro y mantenible que repartir `margin` a mano.

Grid y HTML

Grid organiza visualmente, pero no sustituye a una buena estructura HTML.

🔥 Regla importante

Grid sirve para distribuir visualmente una interfaz.

No debe usarse para ocultar una mala estructura HTML ni para justificar un documento semánticamente pobre.

Referencias del apartado 2

- MDN Web Docs. *Basic concepts of grid layout*.
 - MDN Web Docs. *gap*.
-

3. Columnas, filas y reparto del espacio

Aquí empieza una de las partes más útiles de Grid: decidir cómo se reparte el espacio.

`grid-template-columns`

Permite definir las columnas de la rejilla.

```
.layout {  
  display: grid;  
  grid-template-columns: 1fr 1fr 1fr;  
}
```

Eso crea tres columnas del mismo tamaño.

`grid-template-rows`

Permite definir las filas.

```
.layout {  
  display: grid;  
  grid-template-rows: auto auto;  
}
```

No siempre tendrás que definir filas explícitamente, pero conviene entender que también forman parte de la estructura.

La unidad `fr`

`fr` significa fracción del espacio disponible.

```
.layout {  
  display: grid;  
  grid-template-columns: 2fr 1fr;  
}
```

Aquí la primera columna ocupa aproximadamente el doble que la segunda.

Idea práctica

`fr` resulta especialmente útil porque reparte espacio de forma flexible sin obligarte a depender tanto de anchos fijos.

`repeat()`

Evita repetir varias veces la misma definición.

```
.layout {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
}
```

Esto equivale a escribir tres veces `1fr`, pero de forma más limpia.

`minmax()`

Permite definir un mínimo y un máximo para una pista de la rejilla.

```
.cards {  
  display: grid;  
  grid-template-columns: repeat(3, minmax(220px, 1fr));  
}
```

Esto significa: cada columna no debe bajar de 220px, pero puede crecer hasta ocupar una fracción del espacio disponible.

`minmax()` es muy útil en:

- tarjetas;
- paneles;
- listados;
- y cualquier layout donde quieras cierto control sin rigidez total.

Medidas combinadas

También puedes mezclar anchos fijos y flexibles.

```
.dashboard {  
  display: grid;
```

```
grid-template-columns: 240px 1fr;
}
```

Este patrón es muy común en interfaces con sidebar.

Qué debes retener de esta parte

- `grid-template-columns` define columnas.
- `grid-template-rows` define filas.
- `fr` reparte espacio disponible.
- `repeat()` reduce repetición.
- `minmax()` hace rejillas más flexibles y robustas.
- Grid permite combinar anchos fijos y fluidos con mucha claridad.

Referencias del apartado 3

- MDN Web Docs. *grid-template-columns*.
 - MDN Web Docs. *grid-template-rows*.
 - MDN Web Docs. *repeat()*.
 - MDN Web Docs. *minmax()*.
-

4. Rejillas flexibles con `auto-fit` y `auto-fill`

Esta parte es especialmente útil cuando trabajas con tarjetas o bloques repetidos y quieres que la rejilla se adapte de forma más automática.

`auto-fit`

`auto-fit` intenta encajar tantas columnas como sea posible y permite que las existentes crezcan para ocupar mejor el espacio libre.

```
.cards {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 20px;
}
```

Esto es muy útil en:

- grids de tarjetas;
- catálogos;

- listados de cursos;
- paneles de recursos;
- bloques de información repetida.

`auto-fill`

`auto-fill` también repite columnas automáticamente, pero mantiene la lógica de “rellenar” la rejilla incluso cuando sobran huecos.

La diferencia entre `auto-fit` y `auto-fill` no siempre se percibe de inmediato, pero a nivel práctico conviene recordar esta idea:

- `auto-fit` suele ser más cómodo cuando quieres que las columnas visibles se expandan;
- `auto-fill` resulta útil cuando quieres mantener una lógica más rígida de rejilla.

Criterio práctico

En muchos casos de tarjetas responsive, `auto-fit` con `minmax()` es una de las combinaciones más útiles y productivas de todo Grid.

Qué debes retener de esta parte

- `auto-fit` y `auto-fill` ayudan a crear rejillas más fluidas.
- Combinados con `minmax()`, permiten construir grids muy adaptables con poco CSS.
- Son especialmente útiles en interfaces con contenido repetido.

Referencias del apartado 4

- MDN Web Docs. *repeat()*.
- MDN Web Docs. *Auto-placement in grid layout*.

5. Composición con `grid-template-areas`

Hasta ahora has distribuido columnas y filas por medidas. `grid-template-areas` permite dar un nombre a las zonas del layout y pensarlo de forma más visual.

Ejemplo:

```
.page {  
  display: grid;  
  grid-template-columns: 240px 1fr;
```

```
grid-template-areas:  
  "sidebar main";  
gap: 24px;  
}  
  
.sidebar {  
  grid-area: sidebar;  
}  
  
.main {  
  grid-area: main;  
}
```

Esto resulta útil porque hace más legible la estructura general del layout.

También funciona bien en composiciones más amplias:

```
.page {  
  display: grid;  
  grid-template-columns: 240px 1fr;  
  grid-template-areas:  
    "header header"  
    "sidebar main"  
    "footer footer";  
}
```

Aquí ya puedes pensar la interfaz como un conjunto de zonas:

- cabecera;
- barra lateral;
- contenido principal;
- pie de página.

Valor de `grid-template-areas`

No solo sirve para maquetar. También ayuda a **pensar y comunicar** la estructura del layout.

Qué debes retener de esta parte

- `grid-template-areas` permite nombrar zonas del layout.
- Hace más legible la composición general.

- Funciona especialmente bien en layouts de página o secciones grandes.

Referencias del apartado 5

- MDN Web Docs. *grid-template-areas*.
 - MDN Web Docs. *Realizing common layouts using grids*.
-

6. Posicionamiento y capas

No todos los problemas de una interfaz se resuelven con Grid o Flexbox. A veces necesitas desplazar, fijar o superponer elementos.

Aquí entra `position`.

`position: relative`

Permite desplazar un elemento respecto a su posición normal y, sobre todo, establecer un contexto de referencia para hijos posicionados absolutamente.

```
.card {  
  position: relative;  
}
```

`position: absolute`

Saca el elemento del flujo normal y lo posiciona respecto al ancestro posicionado más cercano.

```
.badge {  
  position: absolute;  
  top: 12px;  
  right: 12px;  
}
```

Esto resulta muy útil en:

- insignias;
- botones flotantes dentro de tarjetas;
- etiquetas superpuestas;
- iconos dentro de campos.

`position: fixed`

Fija un elemento respecto a la ventana del navegador.

```
.help-button {  
  position: fixed;  
  bottom: 20px;  
  right: 20px;  
}
```

`position: sticky`

Hace que un elemento se comporte como normal hasta cierto punto del scroll, y después quede “pegado”.

```
.sidebar {  
  position: sticky;  
  top: 16px;  
}
```

Esto resulta útil en:

- barras laterales;
- cabeceras internas;
- filtros que deben permanecer visibles.

`z-index`

Cuando los elementos se solapan, `z-index` ayuda a controlar qué capa queda por encima.

```
.modal {  
  position: fixed;  
  z-index: 1000;  
}
```

Pero conviene recordar algo importante:

Buen criterio

`z-index` no debe usarse como parche constante.

Si una interfaz necesita valores arbitrarios por todas partes, suele haber un problema de estructura o de apilamiento mal resuelto.

Qué debes retener de esta parte

- `position` resuelve problemas distintos de los de Grid o Flexbox.
- `relative` y `absolute` suelen trabajar juntos.
- `fixed` y `sticky` son muy útiles en patrones de interfaz reales.
- `z-index` controla capas, pero debe usarse con criterio.

Referencias del apartado 6

- MDN Web Docs. *position*.
 - MDN Web Docs. *z-index*.
 - MDN Web Docs. *Understanding stacking context*.
-

7. Imágenes dentro del layout

Las imágenes no solo se insertan: también deben encajar bien dentro de tarjetas, cabeceras, bloques destacados o rejillas.

Dos propiedades resultan especialmente útiles aquí: `object-fit` y `aspect-ratio`.

`object-fit`

Controla cómo se adapta una imagen a su caja.

```
.card img {  
  width: 100%;  
  height: 180px;  
  object-fit: cover;  
}
```

Valores frecuentes:

- `cover`
- `contain`

`cover` suele ser útil en tarjetas o miniaturas, porque llena el espacio aunque recorte parte de la imagen.

`contain` conserva la imagen completa, pero puede dejar huecos.

`aspect-ratio`

Permite fijar una proporción sin depender de hacks más antiguos.

```
.media {  
  aspect-ratio: 16 / 9;  
}
```

Esto resulta muy útil en:

- miniaturas;
- tarjetas con imagen;
- banners;
- previsualizaciones;
- bloques donde quieres uniformidad visual.

Idea práctica

`aspect-ratio` y `object-fit` funcionan muy bien juntos cuando necesitas que varias imágenes diferentes se vean coherentes dentro de una rejilla.

Imágenes reales frente a fondos

También conviene distinguir entre:

- usar una imagen real con `<img>` ;
- usar una imagen como fondo con `background-image` .

En general:

- usa `<img>` cuando la imagen forma parte del contenido;
- usa `background-image` cuando es puramente decorativa o parte del acabado visual.

Referencias del apartado 7

- MDN Web Docs. *object-fit*.
- MDN Web Docs. *aspect-ratio*.
- MDN Web Docs. *background-image*.

8. Responsive design y media queries

Una interfaz responsive es una interfaz capaz de adaptarse a distintos tamaños de pantalla y contextos de visualización sin perder claridad ni usabilidad.

Eso implica mucho más que “hacer que quepa todo”. Implica decidir cómo cambian:

- los layouts;
- los anchos;
- los espacios;
- el número de columnas;
- el tamaño del texto;
- y, a veces, incluso la prioridad visual del contenido.

Qué significa realmente que una interfaz sea responsive

Responsive design no consiste en crear una “versión móvil” completamente aparte y otra “versión escritorio” diferente. Lo habitual hoy es construir una interfaz flexible que pueda reorganizarse de manera progresiva.

Esto puede afectar, por ejemplo, a:

- una rejilla de tarjetas;
- una cabecera con navegación;
- un formulario a varias columnas;
- una barra lateral que desaparece o cambia de posición;
- el tamaño de títulos y bloques destacados.

Enfoque mobile-first

Una estrategia muy extendida y útil consiste en pensar primero la interfaz para pantallas pequeñas y, a partir de ahí, ampliarla.

Eso obliga a:

- priorizar contenido;
- evitar rigidez innecesaria;
- y tomar mejores decisiones de estructura.

Idea práctica

Mobile-first no significa “solo móvil”.

Significa empezar por el escenario más restringido y ampliar después con criterio.

Media queries

Las media queries permiten aplicar reglas solo cuando se cumple una determinada condición. Lo más habitual es trabajar con el ancho de la pantalla.

Ejemplo:

```
.cards {
  display: grid;
  grid-template-columns: 1fr;
  gap: 16px;
}

@media (min-width: 768px) {
  .cards {
    grid-template-columns: repeat(2, 1fr);
  }
}

@media (min-width: 1024px) {
  .cards {
    grid-template-columns: repeat(3, 1fr);
  }
}
```

Aquí el mismo bloque pasa de una columna a dos, y luego a tres.

Qué suele cambiar con media queries

Elemento	Cambios frecuentes
Layout general	Número de columnas o zonas
Tarjetas	Rejilla o anchura
Navegación	Dirección, separación o visibilidad
Formularios	Paso de una columna a varias
Tipografía	Tamaños más amplios o más contenidos
Espaciado	Márgenes y paddings

Media features útiles

Además del ancho, existen otras condiciones que pueden ser relevantes. No necesitas profundizar en todas, pero sí entender que una media query puede responder a varios

contextos.

Las más útiles al empezar suelen ser:

- `width`
- `min-width`
- `max-width`
- `orientation`

Qué debes retener de esta parte

- Responsive design significa adaptar, no duplicar.
- Mobile-first te ayuda a priorizar mejor.
- Las media queries deben responder a problemas reales de layout, no a cambios arbitrarios.
- Muchas decisiones responsive afectan directamente a grids, sidebars, formularios y tarjetas.

Referencias del apartado 8

- MDN Web Docs. *Responsive design*.
 - MDN Web Docs. *Using media queries*.
 - MDN Web Docs. *Media features*.
-

9. Container queries

Las media queries miran el tamaño de la ventana o del dispositivo. Pero a veces lo que te interesa no es la pantalla completa, sino el tamaño del componente o del contenedor donde vive un bloque.

Ahí entran las **container queries**.

Las container queries permiten cambiar el comportamiento de un componente según el tamaño del contenedor en el que está, no según el viewport completo.

Eso es muy útil cuando trabajas con:

- tarjetas reutilizables;
- módulos dentro de dashboard;
- componentes que pueden vivir en columnas anchas o estrechas;
- y diseños más modulares.

Ejemplo conceptual:

```
.card-wrapper {  
  container-type: inline-size;  
}  
  
@container (min-width: 500px) {  
  .card {  
    display: flex;  
    gap: 16px;  
  }  
}
```

La idea importante aquí no es memorizar toda la sintaxis, sino comprender el problema que resuelven:

Idea clave

Las media queries responden al tamaño de la pantalla.

Las container queries responden al tamaño del componente o del contenedor.

No siempre necesitarás container queries en proyectos básicos, pero conviene conocerlas porque tienen mucho sentido en interfaces basadas en componentes.

Qué debes retener de esta parte

- Sirven para adaptar componentes según su espacio disponible.
- Encajan muy bien en interfaces modulares.
- No sustituyen a las media queries: responden a un problema diferente.

Referencias del apartado 9

- MDN Web Docs. *CSS container queries*.
- MDN Web Docs. *Using container size queries*.

10. Patrones de composición real

Todo lo anterior cobra sentido cuando lo aplicas a composiciones que sí aparecen en proyectos reales.

Sidebar + contenido principal

```
.dashboard {  
  display: grid;  
  grid-template-columns: 240px 1fr;  
  gap: 24px;  
}
```

Rejilla de tarjetas adaptable

```
.cards {  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));  
  gap: 20px;  
}
```

Cabecera superior y contenido en dos zonas

```
.page {  
  display: grid;  
  grid-template-columns: 240px 1fr;  
  grid-template-areas:  
    "header header"  
    "sidebar main";  
  gap: 24px;  
}
```

Combinación de Grid y Flexbox

Una interfaz real no suele usar uno “o” otro, sino ambos:

- Grid para la macroestructura;
- Flexbox para el interior de componentes o barras.

Ejemplo:

```
.page {  
  display: grid;  
  grid-template-columns: 260px 1fr;  
  gap: 24px;  
}  
  
.navbar {  
  display: flex;  
  justify-content: space-between;
```

```
align-items: center;
}
```

Combinación de layout y responsive

También es normal combinar Grid con media queries:

```
.page {
  display: grid;
  grid-template-columns: 1fr;
  gap: 20px;
}

@media (min-width: 900px) {
  .page {
    grid-template-columns: 260px 1fr;
  }
}
```

Idea clave

Grid y Flexbox no compiten. Se complementan.

Y el responsive no va aparte del layout: es una parte natural de él.

11. Análisis técnico avanzado y cierre

En esta sesión has empezado a trabajar el layout como sistema. Eso cambia bastante la forma de pensar CSS, porque ya no se trata solo de “mover cajas”, sino de decidir cómo se organiza una interfaz completa y cómo debe adaptarse.

Los errores más frecuentes en esta fase no suelen venir de propiedades aisladas, sino de planteamientos débiles:

- usar Flexbox para todo aunque el problema sea claramente bidimensional;
- usar Grid sin tener claro qué zonas o columnas necesitas;
- abusar de `position` para corregir visualmente lo que el layout no resuelve bien;
- usar `z-index` como parche;
- tratar imágenes sin pensar en proporción, recorte y consistencia;
- o aplicar media queries sin una lógica clara de adaptación.

También es importante recordar que una composición más compleja no es automáticamente mejor. Lo valioso no es hacer layouts espectaculares sin motivo, sino construir interfaces que:

- se entiendan;
- se adapten;
- se mantengan;
- y sirvan al contenido.

Esta sesión te deja preparado para el siguiente paso: interacciones, transiciones, animaciones, efectos más avanzados y organización final del CSS.

Qué debes retener de esta sesión

- Grid es un sistema de layout bidimensional.
- `fr`, `repeat()` y `minmax()` son claves para repartir espacio con flexibilidad.
- `auto-fit` y `auto-fill` ayudan a crear rejillas adaptables.
- `grid-template-areas` hace más legible la composición general.
- `position` resuelve problemas distintos a Grid y Flexbox.
- `z-index` controla capas, pero debe usarse con criterio.
- `object-fit` y `aspect-ratio` ayudan mucho a integrar imágenes en layout.
- Responsive design y media queries forman parte natural de la composición.
- Las container queries ayudan a adaptar componentes según su contenedor.
- Las interfaces reales suelen combinar Grid, Flexbox y adaptación responsive.

Referencias documentales generales

1. MDN Web Docs. *CSS grid layout*.
2. MDN Web Docs. *Basic concepts of grid layout*.
3. MDN Web Docs. *grid-template-columns*.
4. MDN Web Docs. *minmax()*.
5. MDN Web Docs. *grid-template-areas*.
6. MDN Web Docs. *position*.
7. MDN Web Docs. *z-index*.
8. MDN Web Docs. *object-fit*.
9. MDN Web Docs. *aspect-ratio*.
10. MDN Web Docs. *Responsive design*.
11. MDN Web Docs. *Using media queries*.
12. MDN Web Docs. *CSS container queries*.

Mini glosario

Término	Definición breve
Grid	Sistema de layout bidimensional
Rejilla	Estructura de filas y columnas
fr	Fracción del espacio disponible
minmax()	Define un mínimo y un máximo para una pista
auto-fit	Ajusta columnas para ocupar mejor el espacio disponible
grid-template-areas	Permite nombrar zonas del layout
position	Propiedad para controlar la colocación de un elemento
z-index	Controla el orden de apilamiento
object-fit	Ajusta cómo una imagen rellena su caja
aspect-ratio	Define una proporción fija para un elemento
Responsive design	Adaptación de la interfaz a distintos tamaños o contextos
Container query	Regla condicional basada en el tamaño del contenedor

Fragmento de ejemplo integrador

HTML

```
<div class="page">
  <header class="page-header">
    <h1>Panel del bootcamp</h1>
  </header>

  <aside class="page-sidebar">
    <nav>
      <a href="#">Inicio</a>
      <a href="#">Cursos</a>
      <a href="#">Progreso</a>
    </nav>
  </aside>

  <main class="page-main">
    <section class="cards">
```

```

<article class="card">
  
  <h2>HTML y CSS</h2>
  <p>Construye interfaces web limpias y mantenibles.</p>
</article>

<article class="card">
  
  <h2>JavaScript</h2>
  <p>Añade interacción real a tus interfaces.</p>
</article>
</section>
</main>
</div>

```

CSS

```

.page {
  display: grid;
  grid-template-columns: 1fr;
  grid-template-areas:
    "header"
    "main";
  gap: 24px;
}

.page-header {
  grid-area: header;
}

.page-sidebar {
  display: none;
}

.page-main {
  grid-area: main;
}

.cards {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
  gap: 20px;
}

.card img {

```

```
width: 100%;
aspect-ratio: 16 / 9;
object-fit: cover;
}

@media (min-width: 960px) {
  .page {
    grid-template-columns: 240px 1fr;
    grid-template-areas:
      "header header"
      "sidebar main";
  }

  .page-sidebar {
    display: block;
    grid-area: sidebar;
    position: sticky;
    top: 16px;
  }
}
```